Rigtorp vs Vyukov in ImagePipeline: runtime cooperativo e immagini piccole

Seconda parte del confronto delle code nei communicator MoStream — relazione tecnica, 28 ottobre 2026

Sintesi. La prima parte aveva mostrato che, con immagini 512×512 e runtime standard, Rigtorp e Vyukov sono equivalenti. Questa seconda parte estende il confronto in due direzioni.

- **Runtime cooperativo, 512×512** (30 esecuzioni): le due code restano equivalenti, con differenze fra −0,66% e +0,25%.
- **Immagini piccole** (nuovo test con lato configurabile, 200 esecuzioni): nel **runtime standard** con collegamenti 16 → 16 Rigtorp è nettamente superiore quando il messaggio è leggero: **+30,8%** a 16×16, **+17,5%** a 32×32, +5,5% a 64×64. Il vantaggio sparisce da 128×128 in su. Rigtorp è anche molto più **stabile**: a 16×16 la variabilità di Vyukov è del 38,8%, quella di Rigtorp del 2,4%. Con soli 4 thread per stadio, o nel runtime cooperativo, non emerge un vantaggio sistematico di nessuna delle due code.

In sintesi: **Rigtorp conviene nel runtime standard quando molti stadi replicati si scambiano messaggi leggeri**; negli altri casi la scelta della coda non incide sulle prestazioni.

1. Richiamo della prima parte

La scelta della coda dei communicator avviene in compilazione (`MoStream/communication_queue.mojo`): senza definizioni si usa Vyukov, con `-D MOSTREAM_RIGTORP=1` Rigtorp. Ogni esecuzione stampa `{MoStream} Inter-stage queue: …`, che è stato controllato in tutte le esecuzioni riportate qui. Con `test_image_1` (runtime standard, 512×512, 60 s) le mediane differivano di +0,25% (1-2-4-2-1) e +0,35% (1-8-16-8-1).

Ambiente invariato: VM `ungolo@131.114.2.95` (Intel Xeon Gold 5512U, 28 core / 56 thread, 1 nodo NUMA), Mojo 1.1.0 (8189361e), copia di lavoro `~/mostream-rigtorp-bench`, pinning attivo, capacità delle code 1024. Gli hash SHA-256 di tutti i sorgenti della libreria coincidono con quelli dei test di integrazione del 26 ottobre (`Tests/risultati_runtime_queue/20261026-mojo1.1.json`).

2. Runtime cooperativo con immagini 512×512

2.1 Test e condizioni

Si usa `Benchmarks/ImagePipeline/test_image_1_coop.mojo`, identico a `test_image_1` tranne che per `pipeline.run_cooperative(n_workers)` al posto di `pipeline.run()`. Nel runtime cooperativo i communicator usano solo `try_push`/`try_pop`. Sono state misurate tre condizioni:

- **1-2-4-2-1, 10 worker**: un worker per actor, stessa configurazione della prima parte;
- **1-8-16-8-1, 34 worker**: un worker per actor, a 34 thread;
- **1-8-16-8-1, 8 worker**: 34 actor su 8 worker, sovrascrittura 4×, la condizione tipica del runtime cooperativo.

```
cd ~/mostream-rigtorp-bench
mojo build -O3 -I . -I Benchmarks/ImagePipeline \
    Benchmarks/ImagePipeline/test_image_1_coop.mojo -o bin/img1coop_vyukov
mojo build -O3 -I . -I Benchmarks/ImagePipeline -D MOSTREAM_RIGTORP=1 \
    Benchmarks/ImagePipeline/test_image_1_coop.mojo -o bin/img1coop_rigtorp
# uso: ./bin/img1coop_<coda> Source GrayScale Blur Sharpen Sink n_workers
```

2.2 Protocollo

5 ripetizioni da 60 s per coda e condizione, con ordine alternato (Vyukov prima nelle ripetizioni dispari, Rigtorp nelle pari). Rispetto alla prima parte è stata aggiunta **un'esecuzione di riscaldamento scartata**, perché nella prima campagna la primissima esecuzione era risultata anomala. Script `run_coop.sh`, lanciato con `setsid nohup ./run_coop.sh > run_coop.log 2>&1 < /dev/null &`:

```
#!/bin/bash
export MOSTREAM_HOME="$HOME/mostream-rigtorp-bench"
cd "$MOSTREAM_HOME"
OUT=results_coop.csv
echo "config,workers,rep,backend,elapsed_ms,throughput" > $OUT
timeout 300 ./bin/img1coop_vyukov 1 2 4 2 1 10 > warmup_coop.log 2>&1   # riscaldamento,
scartato
for cw in "1 2 4 2 1|10" "1 8 16 8 1|34" "1 8 16 8 1|8"; do
  cfg=${cw%|*}; w=${cw#*|}
  for rep in 1 2 3 4 5; do
    if (( rep % 2 )); then order="vyukov rigtorp"; else order="rigtorp vyukov"; fi
    for b in $order; do
      log=$(timeout 300 ./bin/img1coop_$b $cfg $w 2>&1); rc=$?
      echo "$log" | grep -q "Inter-stage queue: $b" || echo "WARN backend mismatch $b" >
&2
      echo "$log" | grep -q "terminated successfully" || echo "WARN rc=$rc $cfg $w $b" >
&2
      ms=$(echo "$log" | sed -n "s/Elapsed time: \(.*\) ms/\1/p")
      tp=$(echo "$log" | sed -n "s/Throughput: \(.*\) img\/s/\1/p")
      echo "${cfg// /-},$w,$rep,$b,$ms,$tp" | tee -a $OUT
    done
  done
done
echo DONE
```

Tutte le 30 esecuzioni sono terminate correttamente, senza avvisi.

2.3 Risultati

Config.	Worker	Condizione	Vyukov (img/s)	Rigtorp (img/s)	Differenza	Coppie vinte da Rigtorp	CV V / R
1-2-4-2-1	10	10 actor su 10 worker	9.875	9.900	**+0,25%**	5/5	0,2% / 0,2%
1-8-16-8-1	34	34 actor su 34 worker	18.499	18.376	**−0,66%**	0/5	0,5% / 0,3%
1-8-16-8-1	8	34 actor su 8 worker (4×)	12.686	12.690	**+0,04%**	2/5	0,1% / 0,1%

- **Le due code sono equivalenti**: tutte le differenze sono sotto l'1%.
- Con 10 worker Rigtorp vince 5 coppie su 5, con 34 worker le vince tutte Vyukov. Un 5 su 5 per caso ha probabilità 1/16 ($p \approx 0{,}06$, test bilaterale), quindi nessuna delle due tendenze è significativa, ed entrambe sono sotto l'1%.
- Con 34 actor su 8 worker, cioè la condizione più cooperativa, la differenza è +0,04%: il limite sono gli 8 worker che eseguono 34 actor, non le code.

Il runtime cooperativo, a parità di thread, è un po' più lento di quello standard per entrambe le code (mediane Vyukov):

Config.	Standard (parte 1)	Cooperativo	Differenza
1-2-4-2-1	10.161	9.875	−2,8%
1-8-16-8-1	19.696	18.499	−6,1%

3. Perché provare immagini più piccole

Nei benchmark sintetici (`Benchmarks/BenchMark Vyukov vs RigTorp/Campagna_50_ripetizioni`, 50 ripetizioni) Rigtorp vince soprattutto nella **farm** (1 sorgente → N consumatori sulla stessa coda) con **molti consumatori** e **poco lavoro per messaggio**:

FarmThroughput, N consumatori	T = 0 µs	1 µs	2 µs	5 µs
4	0,92	1,44	1,09	1,01
8	1,17	1,56	1,58	1,02
16	1,46	1,92	1,90	1,33

Rapporto di throughput Rigtorp/Vyukov; un valore maggiore di 1 indica un vantaggio di Rigtorp.

Con immagini 512×512 ogni stadio impiega circa 0,1–0,5 ms per messaggio, centinaia di volte oltre la soglia di 5 µs oltre la quale il vantaggio di Rigtorp quasi scompare. Il costo dei filtri cresce con il numero di pixel: riducendo il lato a 16–64 pixel il lavoro per stadio scende nell'ordine dei microsecondi e la comunicazione torna a pesare.

> Il vantaggio sintetico nasce dalle operazioni **bloccanti** di Rigtorp: `push`/`pop` prendono un "biglietto" con un solo `fetch_add` e poi ogni thread attende sul proprio slot. Il `Communicator` usa il `push` bloccante ma, per poter rilevare la fine dello stream, usa `try_pop`, che in Rigtorp è un ciclo CAS simile a quello di Vyukov. Nel runtime

> cooperativo si usano solo `try_push`/`try_pop`. Ci si attende quindi un vantaggio minore di quello sintetico, e concentrato sul lato produttore nel runtime standard.

4. Il nuovo test con dimensione configurabile

I test originali **non sono stati modificati**. Sono state create due copie:

- `Benchmarks/ImagePipeline/test_image_1_small.mojo`, copia di `test_image_1.mojo` (runtime standard);
- `Benchmarks/ImagePipeline/test_image_1_small_coop.mojo`, copia di `test_image_1_coop.mojo` (runtime cooperativo).

In entrambe cambiano solo le costanti di dimensione e durata, ora impostabili in compilazione, e viene aggiunta una stampa di controllo. Differenze rispetto all'originale:

```
-from std.sys import argv
+from std.sys import argv, get_defined_int

-comptime W: Int = 512
-comptime H: Int = 512
-comptime DURATION: Int = 60
+comptime W: Int = get_defined_int["IMG_W", 512]()
+comptime H: Int = get_defined_int["IMG_H", 512]()
+comptime DURATION: Int = get_defined_int["IMG_DURATION", 60]()

+        print("  Image size: " + String(W) + "x" + String(H) + ", duration: " + String
(DURATION) + " s")
```

Senza definizioni i valori restano 512×512 e 60 s, quindi il test si comporta come l'originale. In Mojo 1.1 la funzione si chiama `std.sys.get_defined_int`: il nome `env_get_int`, usato in versioni precedenti, non esiste più. Prima della campagna la funzione è stata verificata con un programma di prova (con `-D IMG_W=32` restituisce 32). Esempio di compilazione ed esecuzione:

```
mojo build -O3 -I . -I Benchmarks/ImagePipeline -D MOSTREAM_RIGTORP=1 \
    -D IMG_W=32 -D IMG_H=32 -D IMG_DURATION=20 \
    Benchmarks/ImagePipeline/test_image_1_small.mojo -o bin/small/img32-rigtorp
./bin/small/img32-rigtorp 1 16 16 16 1
```

Una prova pilota di 3 s a 32×32 con 1-16-16-16-1 aveva dato 619 k img/s per Vyukov e 701 k per Rigtorp nel runtime standard: un primo indizio, da una sola esecuzione, poi confermato dalla campagna.

5. Campagna con immagini piccole

5.1 Protocollo

- lati 16, 32, 64, 128 e 512 (riferimento), immagini quadrate;
- configurazioni **1-4-4-4-1** (14 actor) e **1-16-16-16-1** (50 actor, entro i 56 thread hardware). Fra Grayscale→Blur e Blur→Sharpen i collegamenti sono 4→4 o 16→16, cioè code con molti produttori e molti consumatori;

- runtime standard e cooperativo, quest'ultimo con un worker per actor (14 e 50 worker);
- esecuzioni da 20 s, perché con immagini piccole ogni esecuzione elabora milioni di messaggi; 5 ripetizioni con ordine alternato; un riscaldamento scartato; 20 binari, compilati prima delle misure;
- ogni esecuzione controlla la coda stampata, la dimensione (`Image size: …`) e la terminazione corretta.

Script `run_small.sh` (circa 75 minuti in totale):

```
#!/bin/bash
export MOSTREAM_HOME="$HOME/mostream-rigtorp-bench"
cd "$MOSTREAM_HOME"
SIZES="16 32 64 128 512"; DUR=20
mkdir -p bin/small
for s in $SIZES; do for f in "" _coop; do for q in vyukov rigtorp; do
  D=""; [ $q = rigtorp ] && D="-D MOSTREAM_RIGTORP=1"
  mojo build -O3 -I . -I Benchmarks/ImagePipeline $D -D IMG_W=$s -D IMG_H=$s -D IMG_DURA
TION=$DUR \
    Benchmarks/ImagePipeline/test_image_1_small$f.mojo -o bin/small/img$s$f-$q || { echo
"BUILD FAIL $s$f-$q"; exit 1; }
done; done; done
echo BUILD_OK
sha256sum bin/small/* > bin_small.sha256
OUT=results_small.csv
echo "size,runtime,config,workers,rep,backend,elapsed_ms,throughput" > $OUT
timeout 120 ./bin/small/img32-vyukov 1 16 16 16 1 > warmup_small.log 2>&1   # riscaldame
nto, scartato
for s in $SIZES; do
  for cw in "1 4 4 4 1|14" "1 16 16 16 1|50"; do
    cfg=${cw%|*}; w=${cw#*|}
    for rt in std coop; do
      if [ $rt = coop ]; then suf=_coop; args="$cfg $w"; else suf=""; args="$cfg"; fi
      for rep in 1 2 3 4 5; do
        if (( rep % 2 )); then order="vyukov rigtorp"; else order="rigtorp vyukov"; fi
        for b in $order; do
          log=$(timeout 120 ./bin/small/img$s$suf-$b $args 2>&1); rc=$?
          echo "$log" | grep -q "Inter-stage queue: $b" || echo "WARN backend mismatch
$s $rt $b" >&2
          echo "$log" | grep -q "Image size: ${s}x${s}" || echo "WARN size mismatch $s
$rt $b" >&2
          echo "$log" | grep -q "terminated successfully" || echo "WARN rc=$rc $s $rt $c
fg $b" >&2
          ms=$(echo "$log" | sed -n "s/Elapsed time: \(.*\) ms/\1/p")
          tp=$(echo "$log" | sed -n "s/Throughput: \(.*\) img\/s/\1/p")
          echo "$s,$rt,${cfg// /-},$w,$rep,$b,$ms,$tp" | tee -a $OUT
        done
      done
    done
  done
done
echo DONE
```

Tutti i 20 binari sono stati compilati e tutte le 200 esecuzioni sono terminate correttamente, senza avvisi.

5.2 Risultati: runtime standard, 1-16-16-16-1

Lato	Vyukov (mediana, img/s)	Rigtorp (mediana, img/s)	Differenza	Coppie vinte da Rigtorp	CV V / R
16×16	1,91 M	2,50 M	**+30,83%**	5/5	38,8% / 2,4%
32×32	1,53 M	1,80 M	**+17,49%**	5/5	18,7% / 0,3%
64×64	783,2 k	826,2 k	**+5,49%**	3/5	7,3% / 0,1%
128×128	278,0 k	281,2 k	+1,14%	5/5	1,6% / 0,7%
512×512	20,2 k	20,3 k	+0,65%	5/5	2,7% / 0,6%

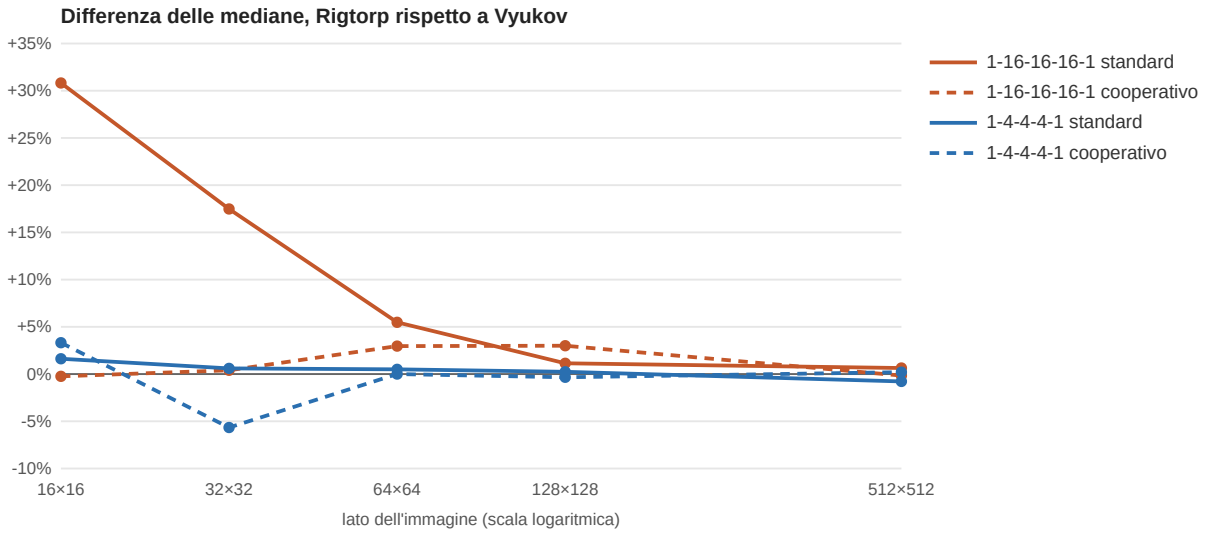


Figura 1 — Differenza percentuale fra le mediane di Rigtorp e Vyukov, per lato dell'immagine, configurazione e runtime. Valori positivi indicano un vantaggio di Rigtorp. Solo la configurazione 1-16-16-16-1 con runtime standard si discosta in modo netto dallo zero.

5.3 Stabilità: le singole ripetizioni

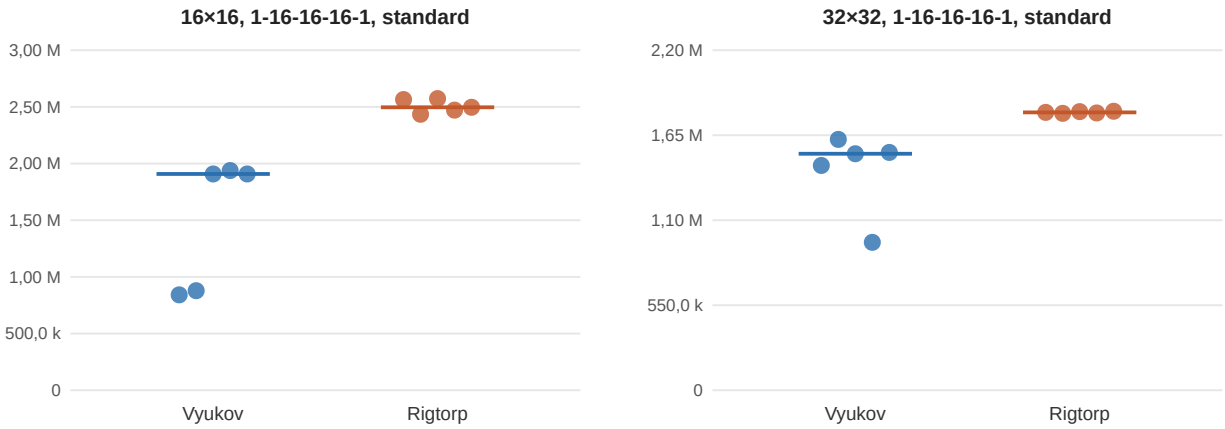


Figura 2 — Throughput delle 5 ripetizioni (punti) e mediana (segmento) per 1-16-16-16-1, runtime standard. Asse verticale da zero.

A 16×16 Vyukov ha un comportamento **bimodale**: due esecuzioni intorno a 859,6 k img/s e tre intorno a 1,92 M img/s. Rigtorp resta fra 2,44 M e 2,57 M img/s in tutte le ripetizioni, e la sua mediana supera di +29% anche la *migliore* esecuzione di Vyukov. Anche a 32×32 Vyukov ha un'esecuzione molto più lenta delle altre. Una spiegazione plausibile, non verificata con strumenti di profilazione, è la diversa gestione della contesa con 16 produttori sulla stessa coda: in Vyukov ogni `push` ritenta un CAS su `enqueue_pos` finché non ha successo, mentre in Rigtorp ogni `push` prende il proprio slot con un solo `fetch_add` e non ritenta.

5.4 Risultati: gli altri casi

Runtime cooperativo, 1-16-16-16-1 (50 worker)

Lato	Vyukov (mediana, img/s)	Rigtorp (mediana, img/s)	Differenza	Coppie vinte da Rigtorp	CV V / R
16×16	151,9 k	151,5 k	−0,24%	2/5	1,7% / 0,8%
32×32	152,4 k	153,0 k	+0,40%	3/5	1,4% / 1,9%
64×64	149,6 k	154,0 k	+2,96%	5/5	1,3% / 1,3%
128×128	150,5 k	155,0 k	+2,99%	3/5	2,2% / 2,2%
512×512	15,9 k	15,8 k	−0,16%	2/5	0,2% / 0,2%

Runtime standard, 1-4-4-4-1

Lato	Vyukov (mediana, img/s)	Rigtorp (mediana, img/s)	Differenza	Coppie vinte da Rigtorp	CV V / R
16×16	1,45 M	1,48 M	+1,62%	5/5	0,4% / 0,2%
32×32	737,4 k	741,8 k	+0,61%	4/5	0,1% / 0,3%
64×64	331,5 k	333,1 k	+0,50%	5/5	0,2% / 0,1%
128×128	127,4 k	127,7 k	+0,25%	3/5	0,4% / 0,2%
512×512	12,0 k	11,9 k	−0,78%	0/5	0,2% / 0,3%

Runtime cooperativo, 1-4-4-4-1 (14 worker)

A 16×16 e 32×32 questa condizione è rumorosa per entrambe le code: a 32×32 sia Vyukov sia Rigtorp hanno un'esecuzione intorno a 410 k img/s contro circa 280 k delle altre (CV 17–20%). Il −5,7% a 32×32, con 2 coppie su 5, non indica quindi una tendenza.

Lato	Vyukov (mediana, img/s)	Rigtorp (mediana, img/s)	Differenza	Coppie vinte da Rigtorp	CV V / R
16×16	262,3 k	271,0 k	+3,33%	4/5	4,6% / 3,9%
32×32	296,5 k	279,7 k	**−5,65%**	2/5	17,0% / 19,8%
64×64	254,5 k	254,5 k	+0,01%	2/5	0,5% / 0,9%
128×128	114,8 k	114,4 k	−0,35%	2/5	0,4% / 0,3%
512×512	11,5 k	11,5 k	+0,20%	3/5	0,5% / 0,5%

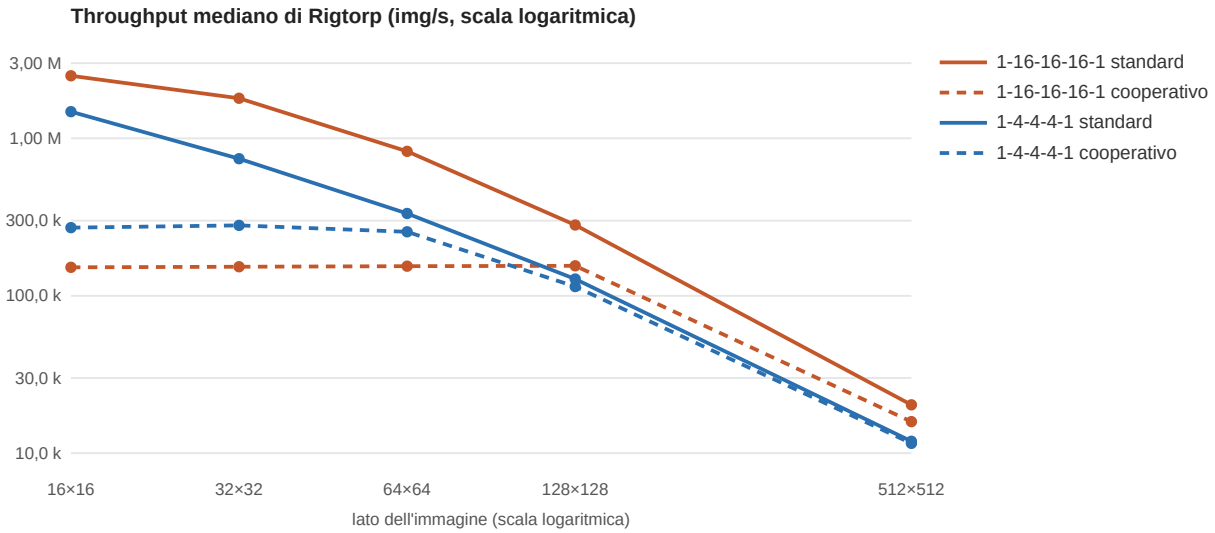


Figura 3 — Throughput mediano di Rigtorp per lato dell'immagine (le curve di Vyukov sono quasi sovrapposte, tranne 1-16-16-16-1 standard a 16 e 32). Il runtime cooperativo con 50 worker si ferma intorno a 150 k img/s qualunque sia la dimensione fino a 128×128.

6. Analisi

6.1 Quando vince Rigtorp

Rigtorp ha un vantaggio netto solo quando valgono **tutte** queste condizioni:

1. **molti thread sulla stessa coda**: 16 produttori e 16 consumatori. Con 4→4, nel runtime standard, le differenze restano fra −0,8% e +1,6% a tutte le dimensioni;
2. **messaggi leggeri**: il vantaggio è +30,8% a 16×16, +17,5% a 32×32, +5,5% a 64×64, poi scende sotto l'1,2%;
3. **runtime standard**: è quello in cui il `Communicator` usa il `push` bloccante di Rigtorp.

L'andamento ricalca quello della farm sintetica: il vantaggio cresce con la contesa e diminuisce con il lavoro per messaggio. L'entità massima osservata (+31%) è inferiore a quella sintetica (fino a 1,9×), coerentemente con l'uso di `try_pop` sul lato consumatore. Oltre al throughput, Rigtorp offre una **prevedibilità** molto maggiore: nelle condizioni di forte contesa la sua variabilità resta fra 0,1% e 2,4%, mentre quella di Vyukov arriva al 38,8%.

6.2 Perché il runtime cooperativo non mostra differenze

Con 50 worker il throughput cooperativo resta intorno a 150 k img/s a 16, 32, 64 e 128 pixel, mentre il runtime standard nelle stesse condizioni arriva a 2,5 M img/s a 16×16. Se il throughput non aumenta quando il lavoro per immagine diminuisce, il collo di bottiglia è altrove: con ogni probabilità nello scheduler cooperativo, cioè nella gestione delle ready/wait queue e nel cambio fra actor, che resta Vyukov per entrambe le configurazioni. In queste condizioni la coda dei communicator incide pochissimo, e infatti le differenze sono fra −0,2% e +3%. L'unico gruppo con una tendenza ripetuta è 64×64 (+3,0%, 5 coppie su 5), un vantaggio piccolo.

6.3 Le immagini 512×512 come controllo

A 512×512 le quattro condizioni danno differenze fra −0,8% e +0,7%, coerenti con la prima parte (+0,25% e +0,35%) e con la campagna cooperativa. La riproducibilità del risultato di parità, con test, durate e configurazioni diverse, rafforza la conclusione che le differenze osservate a 16–64 pixel dipendono dal peso relativo della comunicazione, non da altri fattori della campagna.

7. Conclusioni

- **Runtime standard, molti stadi replicati, messaggi leggeri** (lavoro per messaggio nell'ordine dei microsecondi): conviene **Rigtorp**. È fino al 31% più veloce e molto più stabile.
- **Messaggi pesanti** (immagini da 128×128 in su), **poca contesa** (4 thread per lato) oppure **runtime cooperativo**: le due code sono **equivalenti**. Rigtorp non introduce regressioni misurabili in nessuno dei casi provati.
- Rigtorp è quindi una scelta sicura come coda dei communicator: dove non migliora le prestazioni non le peggiora.

8. Limiti

- **Campione piccolo:** 5 ripetizioni per gruppo bastano per le differenze nette (+31%, +17%), non per quelle sotto il 3%. Sono 20 gruppi confrontati senza correzione per confronti multipli.
- **Spiegazioni non verificate:** il ruolo dei CAS ripetuti in Vyukov e quello dello scheduler cooperativo sono ipotesi coerenti con i dati, non misurate con contatori hardware o strumenti di profilazione.
- **Correttezza:** il sink conta le immagini ma non verifica unicità né contenuto. La correttezza dei communicator è coperta dai test di integrazione (`Tests/run_runtime_queue_tests.py`), non da queste campagne.
- **Ambito:** un solo workload (`test_image_1` e la variante cooperativa), immagini quadrate, capacità delle code 1024, una sola macchina. Nel runtime cooperativo è stato misurato solo un worker per actor nella campagna con immagini piccole.
- **Allocazione:** ogni immagine è un buffer allocato sull'heap dalla sorgente e liberato da un altro thread. Con immagini piccole anche l'allocatore contribuisce al costo per messaggio, allo stesso modo per entrambe le code.

9. File

Percorso	Contenuto
Benchmarks/ImagePipeline/test_image_1_small.mojo	nuovo test, runtime standard, dimensione e durata configurabili
Benchmarks/ImagePipeline/test_image_1_small_coop.mojo	nuovo test, runtime cooperativo
docs/Relazione_ImagePipeline_Rigtorp/results_coop.csv	dati grezzi della campagna cooperativa (30 esecuzioni)
docs/Relazione_ImagePipeline_Rigtorp/results_small.csv	dati grezzi della campagna immagini piccole (200 esecuzioni)
docs/Relazione_ImagePipeline_Rigtorp/ImagePipeline_Rigtorp_vs_Vyukov.pdf	prima parte (runtime standard, 512×512)
VM: ~/mostream-rigtorp-bench/	`run_coop.sh`, `run_small.sh`, log, CSV, binari in `bin/` e `bin/small/`, hash in `bin_small.sha256`

Appendice A — Dati grezzi della campagna cooperativa (512×512)

Config.	Worker	Rip.	Prima in ordine	Vyukov (img/s)	Rigtorp (img/s)	R vs V
1-2-4-2-1	10	1	Vyukov	9.896	9.900	+0,04%
1-2-4-2-1	10	2	Rigtorp	9.875	9.937	+0,63%
1-2-4-2-1	10	3	Vyukov	9.891	9.905	+0,14%
1-2-4-2-1	10	4	Rigtorp	9.844	9.891	+0,48%
1-2-4-2-1	10	5	Vyukov	9.847	9.897	+0,50%
1-8-16-8-1	34	1	Vyukov	18.439	18.372	−0,37%
1-8-16-8-1	34	2	Rigtorp	18.499	18.422	−0,41%
1-8-16-8-1	34	3	Vyukov	18.505	18.388	−0,63%
1-8-16-8-1	34	4	Rigtorp	18.570	18.376	−1,04%
1-8-16-8-1	34	5	Vyukov	18.322	18.279	−0,24%
1-8-16-8-1	8	1	Vyukov	12.710	12.690	−0,15%
1-8-16-8-1	8	2	Rigtorp	12.673	12.700	+0,22%
1-8-16-8-1	8	3	Vyukov	12.686	12.676	−0,08%
1-8-16-8-1	8	4	Rigtorp	12.671	12.722	+0,40%
1-8-16-8-1	8	5	Vyukov	12.691	12.679	−0,10%

Appendice B — Dati grezzi della campagna con immagini piccole

Throughput delle ripetizioni 1–5, in ordine, in img/s. Nelle ripetizioni dispari è stata eseguita prima Vyukov, nelle pari prima Rigtorp. Tempi di esecuzione e valori a piena precisione sono in `results_small.csv`.

Lato	Config.	Runtime	Vyukov, rip. 1–5	Rigtorp, rip. 1–5
16	1-16-16-16-1	standard	841,1 k · 878,1 k · 1,91 M · 1,94 M · 1,91 M	2,56 M · 2,44 M · 2,57 M · 2,47 M · 2,50 M
16	1-16-16-16-1	cooperativo	148,7 k · 154,9 k · 153,9 k · 151,9 k · 150,0 k	149,1 k · 152,1 k · 150,5 k · 151,5 k · 152,0 k
16	1-4-4-4-1	standard	1,45 M · 1,45 M · 1,45 M · 1,44 M · 1,45 M	1,48 M · 1,48 M · 1,48 M · 1,47 M · 1,48 M
16	1-4-4-4-1	cooperativo	262,3 k · 269,5 k · 259,3 k · 262,2 k · 289,6 k	268,3 k · 287,0 k · 287,6 k · 265,4 k · 271,0 k
32	1-16-16-16-1	standard	1,45 M · 1,62 M · 1,53 M · 957,4 k · 1,54 M	1,80 M · 1,79 M · 1,80 M · 1,79 M · 1,80 M
32	1-16-16-16-1	cooperativo	151,4 k · 151,6 k · 152,4 k · 156,2 k · 154,9 k	153,0 k · 155,2 k · 149,2 k · 152,0 k · 156,6 k
32	1-4-4-4-1	standard	737,4 k · 739,1 k · 738,2 k · 736,6 k · 736,9 k	737,3 k · 739,9 k · 741,8 k · 743,3 k · 743,0 k
32	1-4-4-4-1	cooperativo	276,4 k · 296,5 k · 410,2 k · 299,8 k · 295,4 k	301,0 k · 265,1 k · 279,7 k · 276,7 k · 413,9 k
64	1-16-16-16-1	standard	744,7 k · 874,5 k · 779,8 k · 873,9 k · 783,2 k	826,5 k · 826,1 k · 826,2 k · 825,3 k · 827,1 k
64	1-16-16-16-1	cooperativo	149,7 k · 149,2 k · 153,9 k · 149,4 k · 149,6 k	153,4 k · 154,0 k · 157,8 k · 152,7 k · 154,1 k
64	1-4-4-4-1	standard	332,1 k · 330,5 k · 331,5 k · 330,7 k · 331,7 k	333,1 k · 333,7 k · 333,1 k · 333,1 k · 333,0 k
64	1-4-4-4-1	cooperativo	255,9 k · 254,0 k · 254,5 k · 254,8 k · 252,5 k	250,5 k · 252,5 k · 255,1 k · 254,5 k · 255,8 k
128	1-16-16-16-1	standard	269,6 k · 280,2 k · 280,1 k · 278,0 k · 277,6 k	276,9 k · 280,8 k · 281,9 k · 281,3 k · 281,2 k
128	1-16-16-16-1	cooperativo	155,4 k · 149,1 k · 150,5 k · 148,5 k · 155,1 k	150,5 k · 155,2 k · 157,1 k · 155,0 k · 149,0 k
128	1-4-4-4-1	standard	127,4 k · 127,6 k · 127,9 k · 126,7 k · 127,3 k	127,7 k · 127,9 k · 127,8 k · 127,4 k · 127,2 k
128	1-4-4-4-1	cooperativo	114,0 k · 115,2 k · 114,8 k · 114,7 k · 114,8 k	114,3 k · 114,3 k · 114,5 k · 115,1 k · 114,4 k
512	1-16-16-16-1	standard	20,4 k · 19,0 k · 20,2 k · 20,1 k · 20,2 k	20,5 k · 20,3 k · 20,3 k · 20,2 k · 20,4 k
512	1-16-16-16-1	cooperativo	15,8 k · 15,9 k · 15,8 k · 15,9 k · 15,9 k	15,8 k · 15,9 k · 15,8 k · 15,9 k · 15,8 k
512	1-4-4-4-1	standard	12,0 k · 12,0 k · 12,0 k · 12,0 k · 12,0 k	11,9 k · 11,9 k · 11,8 k · 11,9 k · 11,9 k
512	1-4-4-4-1	cooperativo	11,6 k · 11,4 k · 11,5 k · 11,5 k · 11,6 k	11,5 k · 11,6 k · 11,5 k · 11,5 k · 11,6 k